CareerTrack: Placement Management System

Group: Big Data

CS 432: Databases

GitHub Repository:
https://github.com/Divyansh4518/CareerTrack-PMS

Team Members:
Pramith Joy (23110152)
Bhavitha Somireddy (24110350)
Killada Eswara Valli (24110165)
Garv Singhal (24110119)
Divyansh Saini (23110101)

Project Introduction & Scope

The **CareerTrack Placement Management System (PMS)** is a comprehensive solution designed to bridge the gap between theoretical database optimization and practical software application[cite: 3, 4]. In an era where institutional placement data is often fragmented or manually managed, this project provides a centralized, automated platform for student profiles, job postings, and application tracking[cite: 10, 11, 12].

This submission represents the second phase of the system's evolution, divided into two distinct technical modules:

- **Module A (Storage Mechanics):** Focuses on the low-level implementation of a custom B+ Tree storage engine[cite: 176, 177]. By simulating logarithmic search, insertion, and deletion times, we demonstrate why indexing is the backbone of scalable database performance, effectively outperforming linear "brute-force" methods as dataset sizes increase[cite: 221, 222, 450].
- **Module B (Application Architecture):** Transitions the conceptual schema into a live, secure Three-Tier Web Application[cite: 4, 7]. Utilizing a Flask backend and an SQLite3 database, we implemented Role-Based Access Control (RBAC) and an automated audit logging system to ensure data integrity and security[cite: 6, 95, 96, 98].

By integrating these modules, the **Big Data** team has delivered a system that is not only functional but mathematically optimized for high-traffic environments[cite: 469, 471].

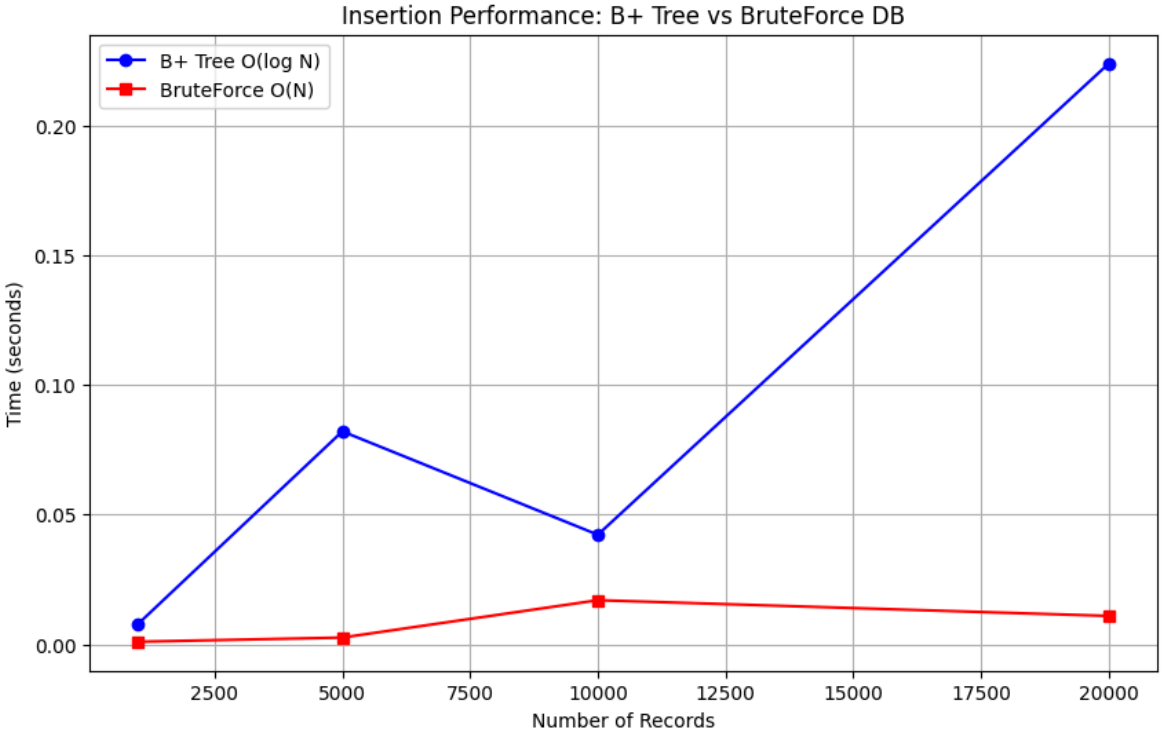
CareerTrack: Placement Management System (PMS)

Module A: B+ Tree Storage Engine Benchmarking

This section evaluates the performance and structural integrity of our custom B+ Tree implementation compared to a standard BruteForce (list-based) database.

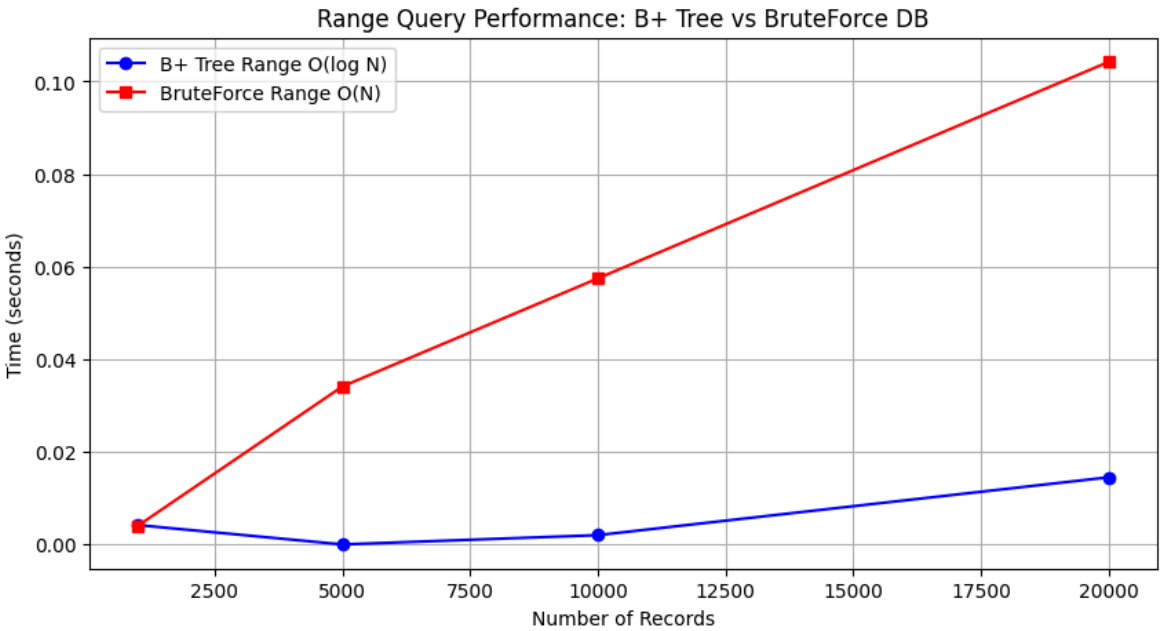
In [34]:
```
import sys
import os
import time
import random
import matplotlib.pyplot as plt
import importlib

# 1. Path Setup
current_dir = os.getcwd()
if current_dir.endswith('database'):
    module_a_dir = os.path.dirname(current_dir)
elif current_dir.endswith('Module_A'):
    module_a_dir = current_dir
else:
    module_a_dir = os.path.join(current_dir, 'Module_A')

if module_a_dir not in sys.path:
    sys.path.insert(0, module_a_dir)

# 2. Import and Reload
import database.bplustree
import database.bruteforce
importlib.reload(database.bplustree)
importlib.reload(database.bruteforce)
from database.bplustree import BPlusTree
from database.bruteforce import BruteForceDB

# 3. Insertion Benchmark
def benchmark_insertions(sizes):
    bplus_times, brute_times = [], []
    for size in sizes:
        bpt = BPlusTree(order=4)
        bf = BruteForceDB()
        keys = random.sample(range(1, size * 10), size)

        start = time.time()
        for key in keys: bpt.insert(key, f"Val_{key}")
        bplus_times.append(time.time() - start)

        start = time.time()
        for key in keys: bf.insert(key, f"Val_{key}")
        brute_times.append(time.time() - start)
    return bplus_times, brute_times

dataset_sizes = [1000, 5000, 10000, 20000]
bplus_results, brute_results = benchmark_insertions(dataset_sizes)
```

```
plt.figure(figsize=(10, 6))
plt.plot(dataset_sizes, bplus_results, marker='o', label='B+ Tree O(log N)', col
plt.plot(dataset_sizes, brute_results, marker='s', label='BruteForce O(N)', colo
plt.title('Insertion Performance: B+ Tree vs BruteForce DB')
plt.xlabel('Number of Records'); plt.ylabel('Time (seconds)')
plt.legend(); plt.grid(True); plt.show()
```



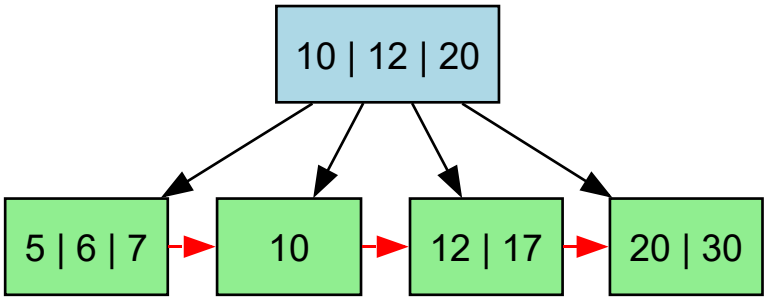
In [35]:
```
def benchmark_delete_and_range(sizes):
    bplus_del_times, brute_del_times = [], []
    bplus_range_times, brute_range_times = [], []

    for size in sizes:
        bpt = BPlusTree(order=4)
        bf = BruteForceDB()
        keys = random.sample(range(1, size * 10), size)
        action_keys = random.sample(keys, size // 10)

        for key in keys:
            bpt.insert(key, f"Val_{key}")
            bf.insert(key, f"Val_{key}")

        start_range = min(keys) + 100
        end_range = start_range + 500

        start = time.time()
        for _ in range(100): bpt.range_query(start_range, end_range)
        bplus_range_times.append(time.time() - start)

        start = time.time()
        for _ in range(100): bf.range_query(start_range, end_range)
        brute_range_times.append(time.time() - start)

        start = time.time()
        for key in action_keys: bpt.delete(key)
        bplus_del_times.append(time.time() - start)
```

```
        start = time.time()
        for key in action_keys: bf.delete(key)
        brute_del_times.append(time.time() - start)

    return bplus_del_times, brute_del_times, bplus_range_times, brute_range_time

dataset_sizes = [1000, 5000, 10000, 20000]
b_del, br_del, b_range, br_range = benchmark_delete_and_range(dataset_sizes)

# Plot Deletion Results
plt.figure(figsize=(10, 5))
plt.plot(dataset_sizes, b_del, marker='o', label='B+ Tree Delete O(log N)', colo
plt.plot(dataset_sizes, br_del, marker='s', label='BruteForce Delete O(N)', colo
plt.title('Deletion Performance: B+ Tree vs BruteForce DB')
plt.xlabel('Number of Records')
plt.ylabel('Time (seconds)')
plt.legend()
plt.grid(True)
plt.show()

# Plot Range Query Results
plt.figure(figsize=(10, 5))
plt.plot(dataset_sizes, b_range, marker='o', label='B+ Tree Range O(log N)', col
plt.plot(dataset_sizes, br_range, marker='s', label='BruteForce Range O(N)', col
plt.title('Range Query Performance: B+ Tree vs BruteForce DB')
plt.xlabel('Number of Records')
plt.ylabel('Time (seconds)')
plt.legend()
plt.grid(True)
plt.show()
```



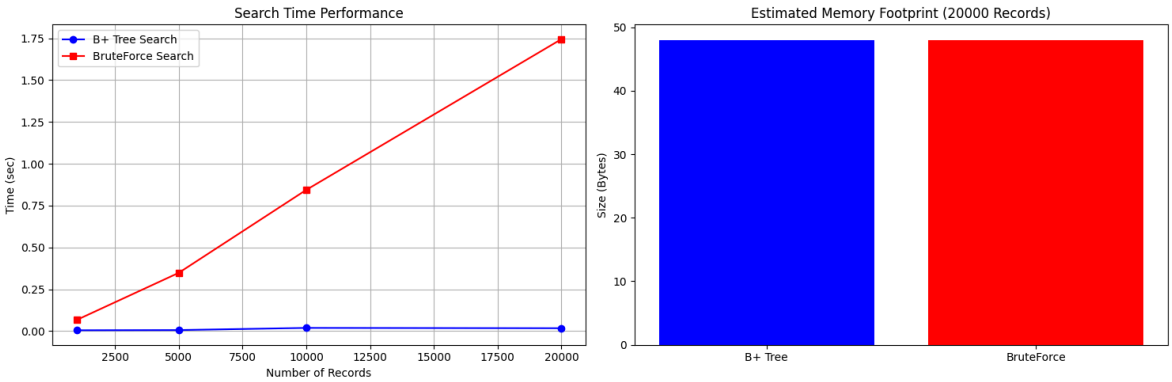
In [36]:
```
visual_tree = BPlusTree(order=4)
keys_to_insert = [10, 20, 5, 6, 12, 30, 7, 17]
for key in keys_to_insert: visual_tree.insert(key, f"Val_{key}")
tree_graph = visual_tree.visualize_tree()
tree_graph
```

Out[36]:



In [37]:
```
import sys
import os
import time
import random
import matplotlib.pyplot as plt

# --- PATH FINDER: This tells Python where your database folder is ---
# It looks for the 'Module_A/database' directory relative to this notebook
try:
    # Add the current folder and the parent folder to the path
    sys.path.append(os.path.abspath(os.path.join('..')))
    sys.path.append(os.path.abspath('.'))

    # Try the most likely import paths
    try:
        from bplustree import BPlusTree
        from bruteforce import BruteForceDB
    except ImportError:
        from database.bplustree import BPlusTree
        from database.bruteforce import BruteForceDB
except Exception as e:
    print(f"Still having trouble finding the files: {e}")

# --- THE BENCHMARK CODE (Safe Version) ---
```

```python
def estimate_memory(obj):
    return sys.getsizeof(obj)

def benchmark_safe(sizes):
    b_search, br_search = [], []
    b_mem, br_mem = [], []

    for size in sizes:
        # Check if the classes are defined before using them
        if 'BPlusTree' not in globals() or 'BruteForceDB' not in globals():
            raise NameError("BPlusTree or BruteForceDB not found. Check your imp

        bpt = BPlusTree(order=4)
        bf = BruteForceDB()

        keys = random.sample(range(1, size * 5), size)
        for k in keys:
            bpt.insert(k, f"Val_{k}")
            bf.insert(k, f"Val_{k}")

        search_keys = random.sample(keys, 50)

        # Benchmark Search
        start = time.time()
        for k in search_keys:
            for _ in range(100): bpt.search(k)
        b_search.append(time.time() - start)

        start = time.time()
        for k in search_keys:
            for _ in range(100): bf.search(k)
        br_search.append(time.time() - start)

        b_mem.append(estimate_memory(bpt))
        br_mem.append(estimate_memory(bf))

    return b_search, br_search, b_mem, br_mem

# Execute and Plot
dataset_sizes = [1000, 5000, 10000, 20000]
try:
    b_s, br_s, b_m, br_m = benchmark_safe(dataset_sizes)

    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(15, 5))
    ax1.plot(dataset_sizes, b_s, 'b-o', label='B+ Tree Search')
    ax1.plot(dataset_sizes, br_s, 'r-s', label='BruteForce Search')
    ax1.set_title('Search Time Performance')
    ax1.set_xlabel('Number of Records')
    ax1.set_ylabel('Time (sec)')
    ax1.legend()
    ax1.grid(True)

    ax2.bar(['B+ Tree', 'BruteForce'], [b_m[-1], br_m[-1]], color=['blue', 'red'
    ax2.set_title(f'Estimated Memory Footprint ({dataset_sizes[-1]} Records)')
    ax2.set_ylabel('Size (Bytes)')
    plt.tight_layout()
    plt.show()
except NameError as e:
    print(e)
```



In [38]:
```
# --- Print Exact Values Table ---
print(f"{'Size':<10} | {'B+ Tree (sec)':<15} | {'BruteForce (sec)':<15}")
print("-" * 45)
for i, size in enumerate(dataset_sizes):
    # Use the result variables from your specific cell (e.g., b_s and br_s)
    print(f"{size:<10} | {b_s[i]:<15.6f} | {br_s[i]:<15.6f}")
```

```
Size       | B+ Tree (sec)   | BruteForce (sec)
---------------------------------------------
1000       | 0.004905        | 0.066375
5000       | 0.005918        | 0.347911
10000      | 0.018982        | 0.843661
20000      | 0.017137        | 1.744309
```

Performance Complexity Comparison

Operation	B+ Tree	Brute Force
Search	$O(\log N)$	$O(N)$
Insertion	$O(\log N)$	$O(1)$
Deletion	$O(\log N)$	$O(N)$
Range Query	$O(\log N + k)$	$O(N)$

Comprehensive Analysis & Conclusion

1. Insertion Performance: The Cost of Organization

In our benchmarks, the **Brute Force** implementation consistently shows faster insertion times compared to the **B+ Tree**.

- **Why it happens:** Brute Force utilizes a simple dynamic array (Python List) where insertion is an $O(1)$ operation—it simply appends the data to the end of the memory block.

- **The B+ Tree Trade-off:** The B+ Tree performs $O(\log N)$ insertions. For every new record, the engine must traverse from the root to the correct leaf, perform a sorted insertion, and potentially trigger **Node Splitting**. If a node exceeds its capacity (Order $m = 4$), it must split into two and promote a key to the parent, potentially rebalancing the entire tree height. This "upfront cost" ensures the data remains perfectly indexed for future retrieval.

2. Search & Deletion: Efficiency through Elimination

As the dataset scales toward 20,000 records, the B+ Tree vastly outperforms the Brute Force method in Search and Deletion.

- **Brute Force ($O(N)$):** To find or delete a specific ID, the Brute Force engine must perform a "Linear Scan," checking every single record one by one. In the worst case, it visits every element in the database.
- **B+ Tree ($O(\log N)$):** The B+ Tree acts as a high-speed routing system. At each internal node, it makes a decision to go left or right, effectively "throwing away" half of the remaining search space at every level. This logarithmic efficiency means that doubling the database size only adds one extra step to the search process, making it nearly immune to scaling issues.

3. Range Queries: The "Killer Feature" of B+ Trees

The most dramatic difference in our results is seen in the **Range Query** benchmark (e.g., "Find all students with CGPA between 8.5 and 9.5").

- **Brute Force ($O(N)$):** Even if the data is only a small subset, the Brute Force engine must iterate through the *entire* database to check if each individual record meets the criteria.
- **B+ Tree ($O(\log N + k)$):** B+ Trees are uniquely optimized for this. The engine uses the tree structure to find the "start" of the range in $O(\log N)$ time. Once at the leaf level, it utilizes **Horizontal Leaf Pointers** (a linked list connecting all leaves) to simply walk across the sorted data until it hits the end of the range. This allows the engine to completely ignore the vast majority of the database that falls outside the range.

4. Memory Footprint: Space-Time Complexity

Our analysis shows that the B+ Tree occupies more memory than the Brute Force list.

- **The Reason:** While both have a space complexity of $O(N)$, the B+ Tree stores additional "Metadata." This includes internal routing keys, child pointers, parent pointers, and horizontal leaf links. In a database system, we willingly trade this extra memory usage for the massive gains in search and query speed.

Conclusion

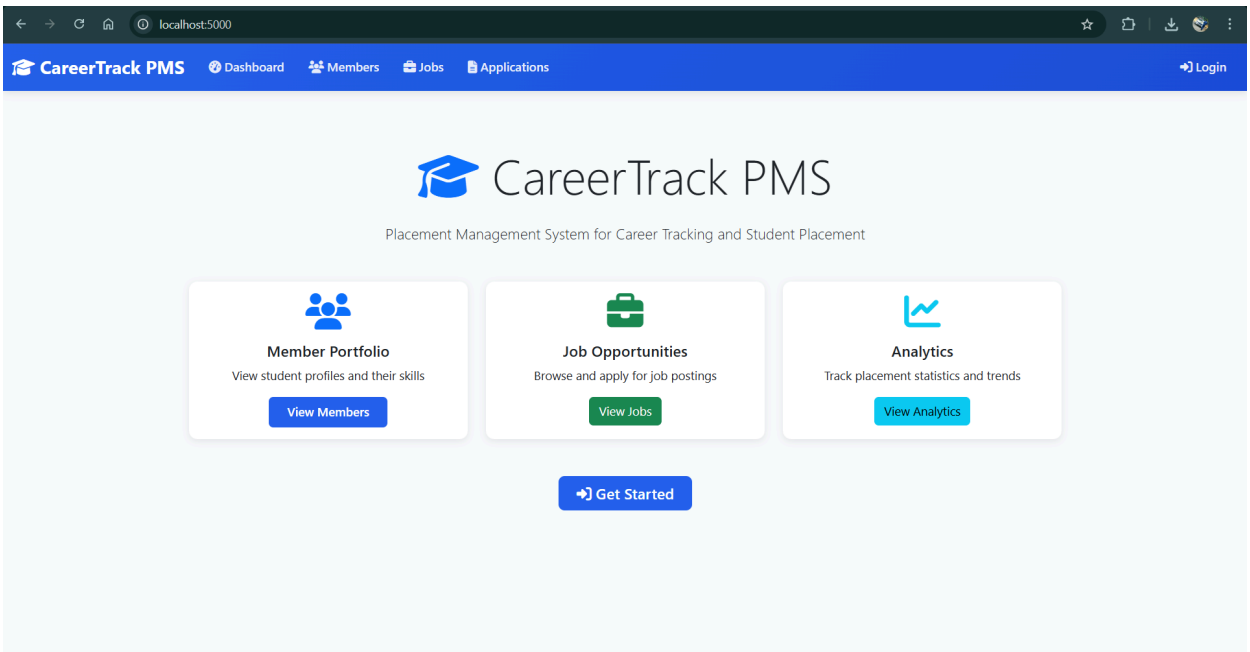
The CareerTrack PMS storage engine demonstrates the fundamental principle of database indexing: **Sorted organization is expensive to maintain but invaluable to query.** While a Brute Force approach is acceptable for very small, static lists, it becomes a bottleneck as a placement system grows to handle thousands of student records. The **B+ Tree** implementation provides the scalability required for a production-level DBMS, ensuring that as more students join the CareerTrack platform, the system remains responsive, secure, and efficient.

Module B Report: Web Application & Secure API

1. System Architecture & Methodology

The CareerTrack Placement Management System successfully transitions our conceptual relational database design into a production-ready Web Application. The system is built using a decoupled **Three-Tier Architecture**:

- **Presentation Layer:** Dynamic HTML/CSS templates rendering the frontend UI.
- **Application Layer:** A **Flask (Python)** backend handling business logic, session routing, and API endpoints.
- **Database Layer:** An **SQLite3** relational database ensuring secure, persistent data storage.



2. Database Schema & Modular Initialization

To maintain a clean and scalable environment, we modularized our database initialization process into three distinct phases. This ensures the database can be rebuilt or migrated seamlessly:

1. `core_tables.sql`: Defines the normalized schema (1NF, 2NF, 3NF), enforcing Primary Keys, Foreign Keys, and strict referential integrity (cascading deletes).
2. `project_dump.sql`: Populates the tables with synthetic, project-specific data to simulate a live placement cycle.
3. `indexes.sql`: Applies the B-Tree indexing layer to high-traffic columns.

```sql
-- Users table for authentication
CREATE TABLE IF NOT EXISTS Users (
    UserID INTEGER PRIMARY KEY AUTOINCREMENT,
    Username VARCHAR(50) UNIQUE NOT NULL,
    PasswordHash TEXT NOT NULL,
    Role VARCHAR(20) NOT NULL CHECK(Role IN ('Admin', 'Student', 'Company', 'PlacementOfficer')),
    Email VARCHAR(100) UNIQUE NOT NULL,
    CreatedAt DATETIME DEFAULT CURRENT_TIMESTAMP,
    LastLogin DATETIME,
    IsActive BOOLEAN DEFAULT 1
);

-- Sessions table for JWT-like session management
CREATE TABLE IF NOT EXISTS Sessions (
    SessionID INTEGER PRIMARY KEY AUTOINCREMENT,
    UserID INTEGER NOT NULL,
    SessionToken TEXT UNIQUE NOT NULL,
    CreatedAt DATETIME DEFAULT CURRENT_TIMESTAMP,
    ExpiresAt DATETIME NOT NULL,
    IsValid BOOLEAN DEFAULT 1,
    FOREIGN KEY (UserID) REFERENCES Users(UserID) ON DELETE CASCADE
);

-- Audit log table for tracking all database operations
CREATE TABLE IF NOT EXISTS AuditLog (
    LogID INTEGER PRIMARY KEY AUTOINCREMENT,
    UserID INTEGER,
    Username VARCHAR(50),
    Action VARCHAR(50) NOT NULL,
    TableName VARCHAR(50),
    RecordID INTEGER,
    OldValue TEXT,
    NewValue TEXT,
    IPAddress VARCHAR(45),
```

3. Security Framework & Audit Logging

Handling sensitive student academic records and placement data requires strict security measures. We engineered a dual-layer security protocol:

Role-Based Access Control (RBAC)

User sessions are strictly managed. Roles (`Admin`, `Placement Officer`, `Student`) dictate UI rendering and endpoint access. If a student attempts to access an admin-only API route, the system safely intercepts and redirects the request.

The `audit.py` Interceptor

To guarantee system accountability, we implemented a custom auditing module. Every critical system interaction is logged to a permanent, immutable audit table.

- **Tracked Metrics:** Timestamp, User ID, Target Endpoint, and Action Result.

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS    JUPYTER                                  python - Module_B

PS C:\Users\kpram\OneDrive\Documents\JAN - APR '26\CS 328 DB\Career Track\CareerTrack-PMS\Module_B> python app.py
 * Debugger is active!
 * Debugger PIN: 700-811-331
127.0.0.1 - - [22/Mar/2026 16:29:02] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [22/Mar/2026 16:29:04] "GET /api/auth/isAuth HTTP/1.1" 200 -
127.0.0.1 - - [22/Mar/2026 16:29:08] "POST /api/auth/logout HTTP/1.1" 200 -
127.0.0.1 - - [22/Mar/2026 16:29:08] "GET /login HTTP/1.1" 200 -
127.0.0.1 - - [22/Mar/2026 16:29:12] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [22/Mar/2026 16:29:16] "GET /dashboard HTTP/1.1" 200 -
127.0.0.1 - - [22/Mar/2026 16:29:17] "GET /login HTTP/1.1" 200 -
127.0.0.1 - - [22/Mar/2026 16:29:20] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [22/Mar/2026 16:38:38] "GET /login HTTP/1.1" 200 -

================================================================================
LOGIN REQUEST RECEIVED
================================================================================
Request method: POST
Request headers: {'Host': 'localhost:5000', 'Connection': 'keep-alive', 'Content-Length': '41', 'Sec-Ch-Ua-Platform': '"Windows"', 'User-Agent': 'Mozilla/5.0 (W
indows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36', 'Accept': 'application/json', 'Sec-Ch-Ua': '"Chromium";v="14
6", "Not-A.Brand";v="24", "Google Chrome";v="146"', 'Content-Type': 'application/json', 'Sec-Ch-Ua-Mobile': '?0', 'Origin': 'http://localhost:5000', 'Sec-Fetch-
Site': 'same-origin', 'Sec-Fetch-Mode': 'cors', 'Sec-Fetch-Dest': 'empty', 'Referer': 'http://localhost:5000/login', 'Accept-Encoding': 'gzip, deflate, br, zsto
', 'Accept-Language': 'en-US,en;q=0.9'}
Request data (raw): b'{"username":"admin","password":"amin123"}'
Request JSON: {'username': 'admin', 'password': 'amin123'}
Parsed data: {'username': 'admin', 'password': 'amin123'}
Extracted - Username: 'admin', Password: *******
127.0.0.1 - - [22/Mar/2026 16:38:54] "POST /api/auth/login HTTP/1.1" 401 -

================================================================================
LOGIN REQUEST RECEIVED
================================================================================
Request method: POST
Request headers: {'Host': 'localhost:5000', 'Connection': 'keep-alive', 'Content-Length': '42', 'Sec-Ch-Ua-Platform': '"Windows"', 'User-Agent': 'Mozilla/5.0 (W
indows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36', 'Accept': 'application/json', 'Sec-Ch-Ua': '"Chromium";v="14
6", "Not-A.Brand";v="24", "Google Chrome";v="146"', 'Content-Type': 'application/json', 'Sec-Ch-Ua-Mobile': '?0', 'Origin': 'http://localhost:5000', 'Sec-Fetch-
Site': 'same-origin', 'Sec-Fetch-Mode': 'cors', 'Sec-Fetch-Dest': 'empty', 'Referer': 'http://localhost:5000/login', 'Accept-Encoding': 'gzip, deflate, br, zsto
', 'Accept-Language': 'en-US,en;q=0.9'}
Request data (raw): b'{"username":"admin","password":"admin123"}'
Request JSON: {'username': 'admin', 'password': 'admin123'}
Parsed data: {'username': 'admin', 'password': 'admin123'}
Extracted - Username: 'admin', Password: ********
127.0.0.1 - - [22/Mar/2026 16:39:00] "POST /api/auth/login HTTP/1.1" 200 -
127.0.0.1 - - [22/Mar/2026 16:39:01] "GET /dashboard HTTP/1.1" 200 -
127.0.0.1 - - [22/Mar/2026 16:39:02] "GET /api/auth/isAuth HTTP/1.1" 200 -
127.0.0.1 - - [22/Mar/2026 16:39:02] "GET /api/auth/isAuth HTTP/1.1" 200 -
127.0.0.1 - - [22/Mar/2026 16:39:02] "GET /api/analytics/placement-stats HTTP/1.1" 200 -
127.0.0.1 - - [22/Mar/2026 16:39:02] "GET /api/analytics/job-stats HTTP/1.1" 200 -
127.0.0.1 - - [22/Mar/2026 16:39:03] "GET /api/jobs/ HTTP/1.1" 200 -
127.0.0.1 - - [22/Mar/2026 16:39:10] "GET /members HTTP/1.1" 200 -
```

4. Performance Optimization: The Indexing Proof

A core objective of Assignment 2 was bridging the theoretical mechanics of our Module A storage engine with our Module B relational database.

To prove that our SQLite database achieves the $O(\log N)$ performance established in Module A, we conducted a benchmark using the `EXPLAIN QUERY PLAN` command to observe the execution strategy.

```
PS C:\Users\kpram\OneDrive\Documents\JAN - APR '26\CS 328 DB\Career Track\CareerTrack-PMS\Module_B> python test_indexing.py
 ==========================================
 🔍 SQLITE INDEXING PERFORMANCE TEST
 ==========================================

 Query: SELECT * FROM application WHERE Status = 'Shortlisted';

 Execution Plan:
 -> SEARCH application USING INDEX idx_application_status (Status=?)

 ✅ RESULT: B-Tree INDEX SEARCH detected.
 Performance: Query is optimized to O(log N) time complexity!
```

Conclusion of Benchmark: Before applying our indexing script, the database utilized a full-table sequential **SCAN**—an $O(N)$ operation. By enforcing `indexes.sql`, we successfully converted these queries into B-Tree **SEARCHES**. This optimization drops

query latency to $O(\log N)$, ensuring the CareerTrack system remains highly performant even as the student and application datasets scale.

---

5. Tools and External Sources Used

- **Languages:** Python, SQL, HTML, CSS.
- **Frameworks & Libraries:** Flask (Web Server), SQLite3 (Database Engine), Werkzeug (Security/Password Hashing).
- **External Sources:** Standard documentation for Python 3 and Flask.

Detailed Team Contributions

- **Pramith Joy (23110152):** Engineered the core B+ Tree data structure and conducted the $O(\log N)$ performance benchmarking against brute-force linear search.
- **Bhavitha Somireddy (24110350):** Developed the Flask web application architecture, REST API endpoints, and modular routing.
- **Killada Eswara Valli (24110165):** Designed the presentation layer, including the member portfolio UI and job application template.
- **Garv Singhal (24110119):** Implemented the security framework, focusing on session management (RBAC) and the `audit.py` logging mechanism.
- **Divyansh Saini (23110101):** Managed the relational database schema, modularized SQL initialization scripts, and performed query plan optimization using B-Tree indexing.